

Mirna Jeannette
Reyes Jiménez
22110212

Junio
2025



Practica 03 - Patrones de diseño paralelo

Materia: computación paralela

Carrera: Tecnologías de Software



Practica 03 - Patrones de diseño paralelo

Introducción

Los patrones de diseño paralelo son estrategias de programación reutilizables que abordan problemas comunes en el desarrollo de software concurrente y paralelo. Permiten a los desarrolladores estructurar algoritmos y programas de manera eficiente para aprovechar múltiples núcleos de procesamiento o máquinas distribuidas, mejorando el rendimiento y la escalabilidad. Estos patrones encapsulan soluciones probadas para la descomposición de tareas, la gestión de datos y la coordinación entre hilos o procesos, facilitando la creación de aplicaciones paralelas robustas y optimizadas.

Patrones de Diseño Paralelo Específicos

Fork-Join

El patrón Fork-Join (Dividir y Unir) es un modelo de computación paralela donde una tarea grande se descompone recursivamente en subtareas más pequeñas e independientes (fork). Estas subtareas se ejecutan en paralelo y, una vez completadas, sus resultados se combinan (join) para producir el resultado final de la tarea original. Es ideal para algoritmos de "divide y vencerás".

¿Cómo funciona?

Fork (Dividir): Una tarea principal se divide en dos o más subtareas que pueden ejecutarse de forma concurrente.

Join (Unir): Los resultados de las subtareas se combinan para reconstruir la solución al problema original. El hilo principal espera a que todas las subtareas forkeadas terminen antes de continuar.

Este proceso puede ser recursivo, donde las subtareas también pueden dividirse en subtareas más pequeñas hasta alcanzar una granularidad manejable que se puede resolver secuencialmente.

Ejemplo:

Cálculo de la suma de un array grande: Se puede dividir el array en dos mitades, y cada mitad se suma en paralelo. Luego, las dos sumas parciales se combinan para obtener la suma total.

En Java, el ForkJoinPool y ForkJoinTask (RecursiveAction y RecursiveTask) son implementaciones de este patrón.

Map

El patrón Map (también conocido como Data Parallelism o parallel for) es un idioma en la computación paralela donde una misma operación simple se aplica a todos los elementos de una secuencia o colección de datos, de forma independiente y potencialmente en paralelo.

¿Cómo funciona?

Se define una función elemental (map function) que toma un elemento de entrada y produce un elemento de salida.

Esta función se aplica a cada elemento del conjunto de datos de entrada.

Dado que las operaciones sobre cada elemento son independientes, pueden ejecutarse simultáneamente en diferentes procesadores o hilos.

No requiere comunicación/sincronización entre las subtareas, excepto una posible barrera al final para asegurar que todas las operaciones hayan concluido.

Ejemplo:

Transformar una lista de números: Si tienes una lista de números y quieres elevar al cuadrado cada número. La operación de elevar al cuadrado es independiente para cada número, permitiendo que se realice en paralelo. [1, 2, 3] -> [1, 4, 9]

Procesamiento de imágenes: Aplicar un filtro de brillo a cada píxel de una imagen.

A menudo se combina con el patrón de Reducción para formar el patrón MapReduce.

Stencil

Las computaciones Stencil (plantilla) son una clase de soluciones de procesamiento de datos numéricos que actualizan los elementos de un array (o cuadrícula) según un patrón fijo, llamado "stencil". La computación del nuevo valor de una celda depende de los valores de sus celdas vecinas inmediatas en iteraciones anteriores.

¿Cómo funciona?

Cada elemento en la cuadrícula (por ejemplo, una matriz 2D o 3D) se actualiza basándose en una "plantilla" que define qué vecinos influyen en su nuevo valor.

Este proceso se repite iterativamente.

La clave para la paralelización es que, en cada iteración, los cálculos para diferentes celdas pueden realizarse en paralelo, siempre y cuando se utilicen los valores de la iteración anterior para todos los vecinos. Esto a menudo requiere sincronización entre iteraciones.

Ejemplo:

Simulaciones de dinámica de fluidos (CFD): Calcular cómo el calor se propaga a través de un material, donde la temperatura de un punto en el tiempo $t+1$ depende de las temperaturas de sus vecinos en el tiempo t .

Procesamiento de imágenes: Aplicar filtros de convolución (como el desenfoque Gaussiano), donde el valor de un píxel se calcula a partir de sus vecinos.

Resolución de ecuaciones diferenciales parciales.

Reducción (Reduction)

El patrón de Reducción (o Fold/Aggregate) implica combinar una colección de objetos a un solo objeto (o a un conjunto más pequeño de objetos) aplicando una operación binaria asociativa (y a menudo conmutativa) de forma repetida.

¿Cómo funciona?

Se divide el conjunto de datos en subconjuntos.

Cada subconjunto se procesa en paralelo, aplicando la operación de reducción de forma local para generar resultados parciales.

Luego, estos resultados parciales se combinan de manera jerárquica (a menudo en forma de árbol de reducción) hasta que se obtiene el resultado final único.

La asociatividad de la operación es crucial porque permite que la combinación se realice en cualquier orden.

Ejemplo:

Sumar todos los elementos de un array: Calcular la suma total de una lista de números. Diferentes partes del array se suman en paralelo, y luego esas sumas parciales se suman entre sí.

Encontrar el valor máximo/mínimo en un conjunto de datos.

Contar la frecuencia de palabras en un texto (común en MapReduce).

Scan (Prefix Sum)

Definición: El patrón Scan (también conocido como Suma de Prefijos o Suma Exclusiva/Inclusiva) toma una secuencia de entrada y produce una secuencia de salida del mismo tamaño, donde cada elemento de la secuencia de salida es el resultado de aplicar una operación binaria asociativa a todos los elementos precedentes (y a veces el propio elemento) en la secuencia de entrada.

¿Cómo funciona?

Para una operación como la suma, una "suma de prefijos inclusiva" de una secuencia $[a_0, a_1, a_2, \dots, a_n]$ produciría $[a_0, a_0+a_1, a_0+a_1+a_2, \dots]$.

Aunque parece secuencial, existen algoritmos paralelos eficientes (a menudo basados en un "árbol" lógico de sumas parciales) que pueden calcular el scan en tiempo logarítmico.

Típicamente implica dos fases: una fase "ascendente" (reduce) para calcular sumas parciales en un árbol, y una fase "descendente" para propagar estas sumas y calcular los resultados finales.

Ejemplo:

Calcular sumas acumuladas: Por ejemplo, para determinar el saldo de una cuenta después de una serie de transacciones.

Generación de histogramas, algoritmos de procesamiento de gráficos.

Fundamental para muchas otras operaciones paralelas, como la compactación de arrays o la asignación de IDs únicos.



Conclusión

Los patrones de diseño paralelo son herramientas esenciales para abordar la complejidad de la programación concurrente. Cada patrón ofrece una solución estructurada a un tipo específico de problema, desde la división recursiva de tareas (Fork-Join) y la aplicación uniforme de operaciones (Map), hasta el cálculo de valores basados en vecinos (Stencil), la agregación de resultados (Reducción) y la generación de sumas acumuladas (Scan). Al comprender y aplicar estos patrones, los desarrolladores pueden escribir código paralelo más legible, mantenible y, crucialmente, eficiente, maximizando el rendimiento en arquitecturas modernas con múltiples núcleos y sistemas distribuidos.

Referencia bibliográfica

Parallel programming: Using the Fork-Join model in Salesforce - West Monroe. (n.d.). =

@Westmonroe. <https://www.westmonroe.com/insights/parallel-programming-using-the-fork-Join-model-in-salesforce>

Reduce (Parallel pattern). (2022, December 5). <https://encyclopedia.pub/entry/37935>